



同濟大學  
TONGJI UNIVERSITY

## 计算机系统结构课程设计实验报告

### 实验一：MIPS 五阶段流水线 CPU 的改造

院系： 计算机科学与技术学院  
专业： 计算机科学与技术  
姓名： 毛星博  
学号： 2351520  
指导老师： 郭玉臣  
完成时间： 2026 年 4 月 2 日

# 一、实验环境和整体介绍

## 1.1 实验环境部署与硬件配置说明

操作系统环境: Windows 11 24H2 64 位

开发环境: vivado2018.3

硬件配置: Xilinx NEXYS4 DDR Artix-7

测试环境: MARS4.5

## 1.2 实验目的

- 回顾 MIPS 五阶段流水线 CPU 的基本结构
- 学习教学用 CPU 各模块的编写以及模块之间的连接结构
- 加深对 Verilog 语言的理解以及对硬件开发的理解
- 为之后移植操作系统做准备工作

## 1.3 实验内容

改造 54 条指令 MIPS 流水线 CPU, 支持到 89 条指令, 实现 CP0、异常处理。

本次实验新增的 35 条指令如下:

- 移动操作指令: movn、movz
- 算术操作指令: clo、madd、maddu、msub、msubu
- 转移指令: b、bal、bgezal、bgtz、blez、bltz、bltzal
- 加载存储指令: ll、lwl、lwr、sc、swl、swr
- 异常相关指令: tge、tgeu、tlr、tlru、tne、teqi、tgei、tgeiu、tlti、tltiu、tnei
- 其他指令: nop、ssnop、sync、pref

本次实验我选择的是上学期自己设计的 CPU, 在原先的基础上, 修改为大端模式, 并将其指令集从原先的 53 条扩展到 89 条。

## 1.4 实验步骤

### (1) 测试程序准备与指令存储初始化

使用 MARS 生成测试程序机器码, 转换为 COE 初始化文件后加载到指令存储器。

### (2) 指令集扩展与控制器译码完善

在控制器中补充新增指令的 opcode/funct 译码与控制信号生成, 覆盖移动类、算术逻辑类、转移分支类、访存类、Trap 类、CP0 相关与特殊指令。完善寄存器写使能、访存使能、写回来源、ALU 操作选择、目的寄存器选择等控制路径。

### (3) 数据通路和执行单元扩展

在执行级和写回级补充新增指令的数据通路, 包括乘法结果写回路径、MDU 相关通路、访存操作类型扩展、CP0 读写通路等, 确保新增指令在 IF-ID-EX-MEM-WB 五级流水中可正确传递与提交。

### (4) CP0、异常与中断机制实现

完善 CP0 的 Status、Cause、EPC、Count、Compare 等关键寄存器行为, 实现 syscall、break、trap、eret 的异常进入与返回流程, 修正异常入口地址及 EPC 保存逻辑。进一步实现 Count/Compare 定时中断触发与中断确认清除流程。

### (5) 仿真验证与错误判定

构建测试平台，按测试程序规则监测 ANSCODE：若出现 -1 到 -10 则定位对应测试函数失败；若低半字为 0x0001 且高半字可随定时中断递增，则判定通过。

#### (6) 下板

下载比特流到板子上，验证数码管显示地址对应的数据单元。最终观测到“低半字显示 0x0001，高半字随时钟中断持续计数”，与测试说明一致。

## 二、指令与数据通路设计

### 2.1 新增的 34 条指令说明

#### 1. movn

- 格式: `movn rd,rs,rt`
- 描述: if  $rt \neq 0$  then  $rd \leftarrow rs$ , 判断地址为  $rt$  的通用寄存器的值, 如果不为零, 那么将地址为  $rs$  的通用寄存器的值赋给地址为  $rd$  的通用寄存器; 反之, 保持地址为  $rd$  的通用寄存器不变。movn 即为 Move Conditional on Not Zero 的意思。
- 部件: PC, NPC, IMEM, RegFile

#### 2. movz

- 格式: `movz rd,rs,rt`
- 描述: if  $rt = 0$  then  $rd \leftarrow rs$ , 与 movn 相反, 判断地址为  $rt$  的通用寄存器的值, 如果为零, 那么将地址为  $rs$  的通用寄存器的值赋给地址为  $rd$  的通用寄存器; 反之, 保持地址为  $rd$  的通用寄存器不变。movz 即为 Move Conditional on Zero 的意思。
- 部件: PC, NPC, IMEM, RegFile

#### 3. clo

- 格式: `clo rd,rs`
- 描述:  $rd \leftarrow \text{coun\_leading\_ones } rs$ , 对地址为  $rs$  的通用寄存器的值, 从其最高位开始向最低位方向检查, 直到遇到值为“0”的位, 将该位之前“1”的个数保存到地址为  $rd$  的通用寄存器中, 如果地址为  $rs$  的通用寄存器的所有位都为 1 (即 0xFFFFFFFF), 那么将 32 保存到地址为  $rd$  的通用寄存器中。
- 部件: PC, NPC, IMEM, RegFile

#### 4. madd

- 格式: `madd rs,rt`
- 描述:  $\{HI, LO\} \leftarrow \{HI, LO\} + rs \times rt$ , 将地址为  $rs$  的通用寄存器的值与地址为  $rt$  的通用寄存器的值作为有符号数进行乘法运算, 运算结果与  $\{HI, LO\}$  相加, 相加的结果保存到  $\{HI, LO\}$  中。此处  $\{HI, LO\}$  表示 HI、LO 寄存器连接形成的 64 位数, HI 是高 32 位, LO 是低 32 位。
- 部件: PC, NPC, IMEM, RegFile, HI, LO

#### 5. maddu

- 格式: `maddu rs,rt`
- 描述:  $\{HI, LO\} \leftarrow \{HI, LO\} + rs \times rt$ , 将地址为  $rs$  的通用寄存器的值与地址为  $rt$  的通用寄存器的值作为无符号数进行乘法运算, 运算结果与  $\{HI, LO\}$  相加, 相加的结果保存到  $\{HI, LO\}$  中。此处  $\{HI, LO\}$  表示 HI、LO 寄存器连接形成的 64 位数, HI 是高 32 位, LO 是低 32 位。
- 部件: PC, NPC, IMEM, RegFile, HI, LO

#### 6. msub

- 格式: `msub rs,rt`
- 描述:  $\{HI, LO\} \leftarrow \{HI, LO\} - rs \times rt$ , 将地址为 `rs` 的通用寄存器的值与地址为 `rt` 的通用寄存器的值作为有符号数进行乘法运算, 运算使用  $\{HI, LO\}$  减去乘法结果, 相减的结果保存到  $\{HI, LO\}$  中。
- 部件: PC, NPC, IMEM, RegFile, HI, LO

#### 7. `msubu`

- 格式: `msubu rs,rt`
- 描述:  $\{HI, LO\} \leftarrow \{HI, LO\} - rs \times rt$ , 将地址为 `rs` 的通用寄存器的值与地址为 `rt` 的通用寄存器的值作为无符号数进行乘法运算, 运算使用  $\{HI, LO\}$  减去乘法结果, 相减的结果保存到  $\{HI, LO\}$  中。
- 部件: PC, NPC, IMEM, RegFile, HI, LO

#### 8. `b`

- 格式: `b offset`
- 描述: 无条件转移, `b` 指令可以认为是 `beq` 指令的特殊情况, 当 `beq` 指令的 `rs`、`rt` 都等于 0 时, 即为 `b` 指令。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 9. `bal`

- 格式: `bal offset`
- 描述: 无条件转移, 并且将转移指令后面第 2 条指令的地址作为返回地址, 保存到通用寄存器 `$31`。`bal` 指令是 `bgezal` 指令的特殊情况, 当 `bgezal` 指令的 `rs` 为 0 时, 就是 `bal` 指令。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 10. `bgezal`

- 格式: `bgezal rs, offset`
- 描述: if  $rs \geq 0$  then branch, 如果地址为 `rs` 的通用寄存器的值大于等于 0, 那么发生转移, 并且将转移指令后面第 2 条指令的地址作为返回地址, 保存到通用寄存器 `$31`。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 11. `bgtz`

- 格式: `bgtz rs, offset`
- 描述: if  $rs > 0$  then branch, 如果地址为 `rs` 的通用寄存器的值大于零, 那么发生转移。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 12. `blez`

- 格式: `blez rs, offset`
- 描述: if  $rs \leq 0$  then branch, 如果地址为 `rs` 的通用寄存器的值小于等于零, 那么发生转移。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 13. `bltz`

- 格式: `bltz rs, offset`
- 描述: if  $rs < 0$  then branch, 如果地址为 `rs` 的通用寄存器的值小于 0, 那么发生转移。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 14. `bltzal`

- 格式: bltzal rs, offset
- 描述: if  $rs < 0$  then branch, 如果地址为 rs 的通用寄存器的值小于 0, 那么发生转移, 并且将转移指令后面第 2 条指令的地址作为返回地址, 保存到通用寄存器 \$31。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT18, ADD

#### 15.11

- 格式: ll rt, offset(base)
- 描述: 从内存中指定的加载地址处, 读取一个字节, 然后符号扩展至 32 位, 保存到地址为 rt 的通用寄存器中。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 16.lwl

- 格式: lwl rt, offset(base)
- 描述: 从内存中指定的加载地址处, 加载一个字的最高有效部分。lwl 指令对加载地址没有要求, 从而允许地址非对齐加载, 这是与 lh、lhu、lw 指令的不同之处。在大端模式、小端模式下, lwl 指令的效果不同。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 17.lwr

- 格式: lwr rt, offset(base)
- 描述: 从内存中指定的加载地址处, 加载一个字的最低有效部分。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 18.sc

- 格式: sc rt, offset(base)
- 描述: 如果 RMW 序列没有受到干扰, 也就是 LLbit 为 1, 那么将地址为 rt 的通用寄存器的值保存到内存中指定的存储地址处, 同时设置地址为 rt 的通用寄存器的值为 1, 设置 LLbit 为 0。如果 RMW 序列受到了干扰, 也就是 LLbit 为 0, 那么不修改内存, 同时设置地址为 rt 的通用寄存器的值为 0。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 19.swl

- 格式: swl rt, offset(base)
- 描述: 将地址为 rt 的通用寄存器的高位部分存储到内存中指定的地址处, 存储地址的最后两位确定了要存储 rt 通用寄存器的哪几个字节。swl 指令对存储地址没有对齐要求, 这是与 sh、sw 指令的不同之处。在大端模式、小端模式下, swl 指令的效果不同。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 20.swr

- 格式: swr rt, offset(base)
- 描述: 将地址为 rt 的通用寄存器的低位部分存储到内存中指定的地址处。
- 部件: PC, NPC, IMEM, RegFile, ALU, EXT16, DMEM

#### 21.tge

- 格式: tge rs, rt
- 描述: if  $GPR[rs] \geq GPR[rt]$  then trap, 将地址为 rs 的通用寄存器的值, 与地址为 rt 的通用寄存器的值作为有符号数进行比较, 如果前者大于等于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**22. tgeu**

- 格式: `tgeu rs, rt`
- 描述: if  $GPR[rs] \geq GPR[rt]$  then trap, 将地址为 `rs` 的通用寄存器的值, 与地址为 `rt` 的通用寄存器的值作为无符号数进行比较, 如果前者大于等于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**23. tlt**

- 格式: `tlt rs, rt`
- 描述: if  $GPR[rs] < GPR[rt]$  then trap, 将地址为 `rs` 的通用寄存器的值, 与地址为 `rt` 的通用寄存器的值作为有符号数进行比较, 如果前者小于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**24. tltu**

- 格式: `tltu rs, rt`
- 描述: if  $GPR[rs] < GPR[rt]$  then trap, 将地址为 `rs` 的通用寄存器的值, 与地址为 `rt` 的通用寄存器的值作为无符号数进行比较, 如果前者小于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**25. tne**

- 格式: `tne rs, rt`
- 描述: if  $GPR[rs] \neq GPR[rt]$  then trap, 将地址为 `rs` 的通用寄存器的值, 与地址为 `rt` 的通用寄存器的值进行比较, 如果不相等, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**26. teqi**

- 格式: `teqi rs, immediate`
- 描述: if  $GPR[rs] = \text{sign\_extended}(\text{immediate})$  then trap, 将地址为 `rs` 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值进行比较, 如果两者相等, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**27. tgei**

- 格式: `tgei rs, immediate`
- 描述: if  $GPR[rs] \geq \text{sign\_extended}(\text{immediate})$  then trap, 将地址为 `rs` 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值作为有符号数进行比较, 如果前者大于等于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU
- 数据通路:

**28. tgeiu**

- 格式: `tgeiu rs, immediate`
- 描述: if  $GPR[rs] \geq \text{sign\_extended}(\text{immediate})$  then trap, 将地址为 `rs` 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值作为无符号数进行比较, 如果前者大于等于后者, 那么引发自陷异常。
- 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

**29. tlti**

- 格式: `tlti rs, immediate`

• 描述: if  $GPR[rs] < sign\_extended(immediate)$  then trap, 将地址为 rs 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值作为有符号数进行比较, 如果前者小于后者, 那么引发自陷异常。

• 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

### 30. tltiu

• 格式: tltiu rs, immediate

• 描述: if  $GPR[rs] < sign\_extended(immediate)$  then trap, 将地址为 rs 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值作为无符号数进行比较, 如果前者小于后者, 那么引发自陷异常。

• 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

### 31. tnei

• 格式: tnei rs, immediate

• 描述: if  $GPR[rs] \neq sign\_extended(immediate)$  then trap, 将地址为 rs 的通用寄存器的值, 与指令中 16 位立即数符号扩展至 32 位后的值进行比较, 如果两者不相等, 那么引发自陷异常。

• 部件: PC, NPC, CP0, IMEM, RegFiles, ALU

### 32. nop

• 描述: 空指令。

### 33. ssnop

• 描述: 一种特殊类型的空指令, 此次实验中实现的 MIPS CPU 中与 nop 的作用相同, 所以可以按照 nop 指令的处理方式来处理 ssnop 指令。

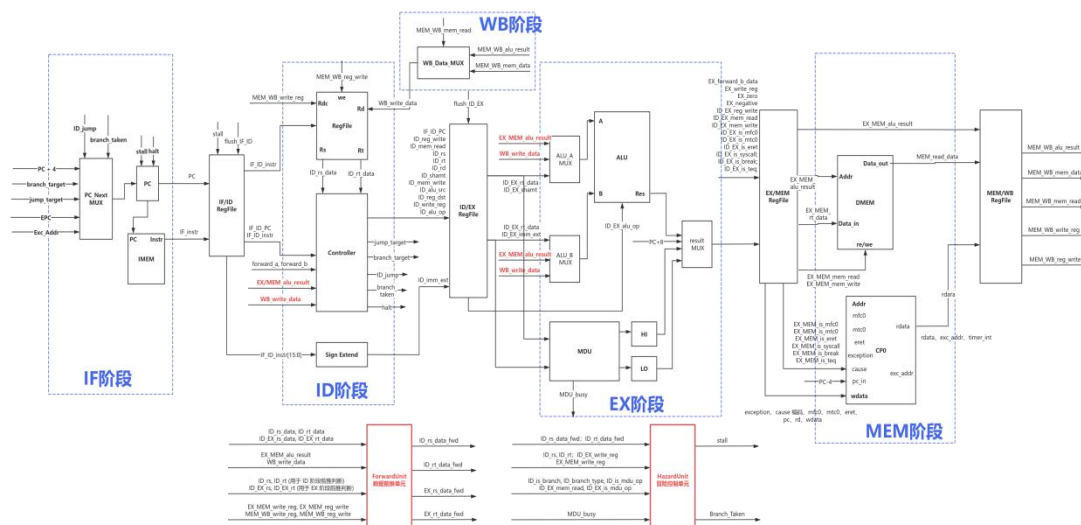
### 34. sync

• 描述: 用于保证加载、存储操作的顺序。此次实验中实现的 MIPS CPU 中, 售加载、存储操作是严格按照指令顺序执行的, 所以可将 sync 指令当作 nop 指令处理。

### 35. pref

• 描述: 用于缓存预取。此次实验中实现的 MIPS CPU 中没有实现缓存, 所以可将 pref 指令当作 nop 指令处理。

## 2.2 整体数据通路



### 三、流水线 CPU 各模块改造说明

#### 3.1 控制器模块改造 (Controller.v)

主要改动：

- 新增 SPECIAL2 指令簇译码 (mul、clz、clo、madd、maddu、msub、msubu)
- 新增 ll、sc、lwl、lwr、swl、swr 访存类控制
- 新增 CP0 与异常相关控制 (mfc0、mtc0、eret、trap\_op)

```
// SPECIAL2 指令译码
wire s2_madd = (opcode == OP_SPECIAL2) && (funct == FUNC_MADD);
wire s2_maddu = (opcode == OP_SPECIAL2) && (funct == FUNC_MADDU);
wire s2_mul = (opcode == OP_SPECIAL2) && (funct == FUNC_MUL);
wire s2_msub = (opcode == OP_SPECIAL2) && (funct == FUNC_MSUB);
wire s2_msubu = (opcode == OP_SPECIAL2) && (funct == FUNC_MSUBU);
wire s2_clz = (opcode == OP_SPECIAL2) && (funct == FUNC_CLZ);
wire s2_clo = (opcode == OP_SPECIAL2) && (funct == FUNC_CLO);

// CP0 指令译码
wire cop0_mfc0 = (opcode == OP_COP0) && (rs == 5'b00000);
wire cop0_mtc0 = (opcode == OP_COP0) && (rs == 5'b00100);
wire cop0_eret = (opcode == OP_COP0) && (funct == FUNC_ERET);

// 写使能、访存与 ALU 源
assign reg_write = (r_add | r_addu | r_sub | r_subu | r_and | r_or | r_xor | r_nor |
                    r_slt | r_sltu | r_sll | r_srl | r_sra | r_sllv | r_srlv | r_srav |
                    r_movn | r_movz | r_mfhi | r_mflo | r_jalr |
                    i_addi | i_addiu | i_andi | i_ori | i_xori | i_slti | i_sltiu | i_lui |
                    is_load | i_sc | j_jal | i_bltzal | i_bgezal | i_bal |
                    cop0_mfc0 | s2_mul | s2_clz | s2_clo) && !is_noop_instr;
assign mem_to_reg = is_load | i_sc;
assign alu_src = i_addi | i_addiu | i_andi | i_ori | i_xori | i_slti | i_sltiu | i_lui |
                is_load | is_store | i_sc | i_teqi | i_tgei | i_tgeiu | i_tlti | i_tltiu | i_tnei;

// 乘除法与异常控制输出
assign is_mdu_op = r_mult | r_multu | r_div | r_divu | s2_madd | s2_maddu | s2_msub | s2_msubu;
assign is_mfc0 = cop0_mfc0;
assign is_mtc0 = cop0_mtc0;
assign is_eret = cop0_eret;
assign is_trap = r_tge | r_tgeu | r_tlt | r_tltu | r_teq | r_tne |
                i_teqi | i_tgei | i_tgeiu | i_tlti | i_tltiu | i_tnei;
```

#### 3.2 顶层流水线数据通路改造 (DCPU.v)

改造目标：将新增指令、异常与中断完整接入 IF/ID/EX/MEM/WB。

主要改动：

- 增加 PC 更新优先级链路（异常优先）
- EX 阶段增加 mul、movn、movz、link 写回路径
- MEM 阶段增加 mem\_op 编码与 ll/sc 结果返回
- 接入 CP0\_exception 与 cause 编码

```

wire exc_occur = CP0_exception && !stall;
wire eret_occur = EX_MEM_is_eret && !stall;

assign PC_next = exc_occur ? 32'h00400004 :
                  eret_occur ? CP0_exc_addr :
                  ID_is_jr ? ID_rs_data_fwd :
                  ID_is_jump ? jump_target :
                  branch_taken ? branch_target :
                  PC_plus_4;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        PC <= 32'h00400000;
    end else if (ena && !stall) begin
        PC <= PC_next;
    end
end

wire EX_is_mul = (ID_EX_opcode == OP_SPECIAL2) && (ID_EX_funcnt == FUNC_MUL);
wire [63:0] EX_mul_res = $signed(EX_rs_data_fwd) * $signed(EX_rt_data_fwd);

wire [31:0] EX_result = ID_EX_is_mfhi ? HI_reg :
                        ID_EX_is_mflo ? LO_reg :
                        (ID_EX_is_movn || ID_EX_is_movz) ? EX_rs_data_fwd :
                        EX_is_mul ? EX_mul_res[31:0] :
                        ID_EX_is_link ? (ID_EX_PC + 8) :
                        EX_alu_result;

EX_MEM_reg_write <= ID_EX_reg_write &&
                    (!ID_EX_is_movn || (EX_rt_data_fwd != 32'b0)) &&
                    (!ID_EX_is_movz || (EX_rt_data_fwd == 32'b0));

wire EX_MEM_is_ll = (EX_MEM_opcode == OP_LL);
wire EX_MEM_is_lwl = (EX_MEM_opcode == OP_LWL);
wire EX_MEM_is_lwr = (EX_MEM_opcode == OP_LWR);
wire EX_MEM_is_sc = (EX_MEM_opcode == OP_SC);
wire EX_MEM_is_swl = (EX_MEM_opcode == OP_SWL);
wire EX_MEM_is_swr = (EX_MEM_opcode == OP_SWR);

wire [3:0] mem_op;
assign mem_op = (EX_MEM_opcode == 6'b100011) ? 4'b0000 :
                ((EX_MEM_opcode == 6'b100000) || EX_MEM_is_ll) ? 4'b0001 :

```

```

        (EX_MEM_opcode == 6'b100100) ? 4'b0010 :
        (EX_MEM_opcode == 6'b100001) ? 4'b0011 :
        (EX_MEM_opcode == 6'b100101) ? 4'b0100 :
        EX_MEM_is_lwl ? 4'b0101 :
        EX_MEM_is_lwr ? 4'b0110 :
        ((EX_MEM_opcode == 6'b101011) || EX_MEM_is_sc) ? 4'b1000 :
        (EX_MEM_opcode == 6'b101000) ? 4'b1001 :
        (EX_MEM_opcode == 6'b101001) ? 4'b1010 :
        EX_MEM_is_sw1 ? 4'b1011 :
        EX_MEM_is_swr ? 4'b1100 : 4'b0000;
wire [31:0] MEM_access_data = EX_MEM_is_sc ? {31'b0, LLbit} : MEM_read_data;

```

### 3.3 数据存储器改造 (DMEM.v)

改造目标：实现大端访存与非对齐访问，并支持板级显示调试输出。

主要改动：

- 按字节通道重构存储阵列，稳定支持大端字节拼接
- 实现 lwl/lwr/swl/swr 语义
- 导出 anscore\_dbg 供顶层数码管显示

```

reg [7:0] mem_b3 [0:255];
reg [7:0] mem_b2 [0:255];
reg [7:0] mem_b1 [0:255];
reg [7:0] mem_b0 [0:255];

wire [31:0] mem_word = {mem_b3[word_addr], mem_b2[word_addr], mem_b1[word_addr],
mem_b0[word_addr]};
assign anscore_dbg = {mem_b3[8'd0], mem_b2[8'd0], mem_b1[8'd0], mem_b0[8'd0]};

```

### 3.4 CP0 协处理器改造 (CP0.v)

改造目标：补齐异常进入/返回、Count/Compare 定时中断链路。

主要改动：

- 异常时写 EPC、Cause 并更新状态位
- Count 自增，Compare 命中后拉起 timer\_pending
- 写 Compare 清中断挂起，eret 恢复状态

```

assign exc_addr = eret ? cp0_regs[14] : 32'h0040_0004;
assign timer_int = timer_pending && cp0_regs[12][0];

else if (exception && cp0_regs[12][0]) begin
    cp0_regs[14] <= pc;
    cp0_regs[13][6:2] <= cause;
    cp0_regs[12] <= cp0_regs[12] << 5;
end

```

```

    if (cause == 5'd0) timer_pending <= 1'b0;
end else if (mtc0) begin
    cp0_regs[Rd] <= wdata;
    if (Rd == 5'd11) timer_pending <= 1'b0;
end else if (eret) begin
    cp0_regs[12] <= cp0_regs[12] >> 5;
end else begin
    cp0_regs[9] <= cp0_regs[9] + 32'd1;
    if ((cp0_regs[11] != 32'b0) && ((cp0_regs[9] + 32'd1) == cp0_regs[11])) begin
        timer_pending <= 1'b1;
    end
end
end

```

### 3.5 板级显示链路改造 (top\_borad.v)

改造目标：满足实验要求，显示 0x10010000 对应 DMEM 单元值。

主要改动：

- 从 DCPU 导出 anscode\_out
- 数码管显示数据为 anscode\_value

```

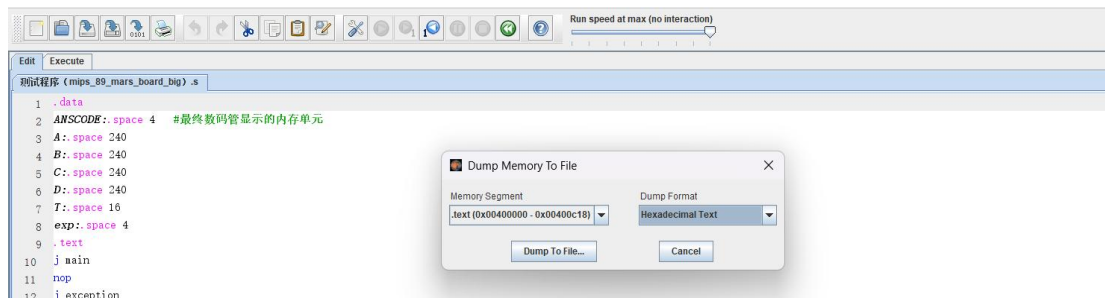
wire [31:0] anscode_value;
DCPU cpu(
    .clk(clk_cpu),
    .rst(rst),
    .ena(ena),
    .intr(1'b0),
    .PC_out(pc_value),
    .instr_out(instr_value),
    .reg28(reg28_value),
    .anscode_out(anscode_value)
);
wire [31:0] display_data = anscode_value;

```

## 四、仿真测试

### 4.1 测试指令 coe 文件

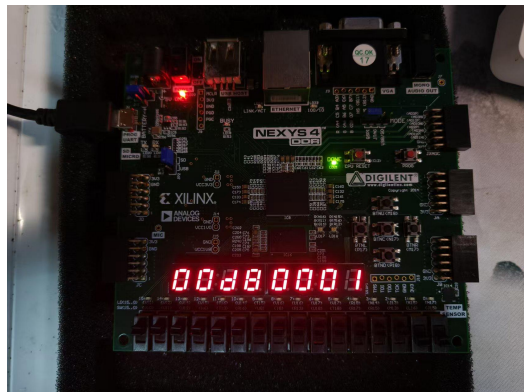
使用 mars 将测试程序转换为 coe 文件，然后导入 IP 核进行初始化

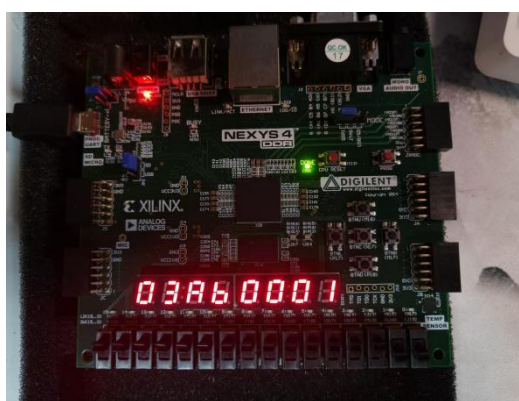
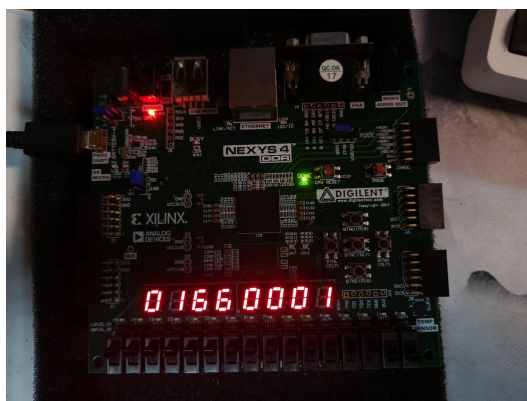


## 4.2 仿真测试文件编写

仿真的 tb 文件首先产生 100 MHz 等效仿真时钟，并在仿真开始阶段施加复位；随后通过 force 将顶层分频后的 CPU 时钟替换为测试平台快时钟，以缩短观察时间。在功能观测方面，测试平台导出两个关键信号：wave\_display\_word（顶层送往数码管的显示字）与 wave\_dmem\_10010000（显示地址 0x10010000 对应的 DMEM 单元值）。仿真运行固定周期后自动结束，并在控制台打印当前 ANSCODE、显示字和 PC 值，便于快速确认结果。

```
module tb_top_board;
    reg clk;
    reg rst;
    reg [1:0] switch;
    localparam integer SIM_CYCLES = 300000;
    wire [7:0] seg;
    wire [7:0] an;
    top_board dut (
        .clk(clk),
        .rst(rst),
        .switch(switch),
        .seg(seg),
        .an(an)
    );
    // Core waveform taps: display word and mapped DMEM cell (0x10010000).
    wire [31:0] wave_display_word = dut.display_data;
    wire [31:0] wave_dmem_10010000 = dut.cpu.dmem.anscode_dbg;
    // 100MHz equivalent simulation clock (10ns period)
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end
    initial begin
        rst = 1'b1;
        switch = 2'b00; // switch[0]=0 run
        // Use fast TB clock for CPU in simulation.
        force dut.clk_cpu = clk;
        #100;
        rst = 1'b0;
        repeat (SIM_CYCLES) @(posedge clk);
        $display("TB_DONE: waveform capture complete, ANSCODE=%h, display=%h, PC=%h",
dut.cpu.dmem.anscode_dbg, dut.display_data, dut.pc_value);
        release dut.clk_cpu;
        $finish;
    end
endmodule
```





## 六、实验总结与感想

在本次实验中，我成功在原先 54 指令集 CPU 上进行改造，使其最终支持 89 条指令，并成功运行测试程序。这个过程并不容易，在中途遇见了很多问题。比如 `movn/movz` 这种看起来简单的条件写回，真到流水线里就会牵扯写使能时机；再比如 `CP0`、异常和中断，单独看逻辑不复杂，但和流水线冲刷、`PC` 优先级放在一起就很容易出错。而且有些问题在普通指令测试里完全看不出来，只有跑到对应测试函数才知道。

一个挺深的体会是“定位问题的方法比硬改代码更重要”。后面我基本是按测试程序错误码、波形和日志三条线去定位，效率明显高了。之前如果只盯着某个模块改，容易越改越乱；后来每次改完都回归一轮自动判定，心里就比较有底。包括最后上板显示这部分，也是先把仿真里的显示链路打通，再去查看硬件现象，少走了很多弯路。

这次还踩了不少工程层面的坑，比如仿真环境变量污染、资源超量导致布局失败。在现阶段，`DMEM` 占用了过多的 `LUT` 导致资源爆掉，这是我之前从未遇到过的场景。在经过多次尝试后，最后通过重构存储器写法才把实现拉回来。这个过程让我对“写可综合代码”和“写可实现代码”的区别感受很深，在写代码的时候，不仅要考虑代码的正确性，还要考虑效率和资源占用，这给了我不少启发。

通过本次实验，我对流水线 CPU 的工作流程的理解进一步加深。从指令扩展、冒险处理、异常中断、仿真验证，到综合实现、下板显示，基本都过了一遍。希望在后续的实验中，我能够学习到更多知识，进一步加深对计算机系统结构的理解。